

## 03\_base-py

#PostgreSQL #python

### 1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ db/  
            └─ base.py
```

El archivo `base.py` se encuentra dentro del módulo de base de datos del backend:

```
backend/app/db/
```

Su función principal es definir la clase base común de los modelos SQLAlchemy del proyecto.

Este archivo es pequeño, pero importante, porque proporciona la clase de la que heredarán los modelos ORM como:

```
Event  
Rule  
Alert
```

La relación general es:

```
base.py  
  ↓  
Base  
  ↓  
models/event.py  
models/rule.py  
models/alert.py  
  ↓  
tablas de PostgreSQL
```

### 2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,220p' backend/app/db/base.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,220p'
```

Indica que se impriman las líneas de la 1 a la 220.

```
backend/app/db/base.py
```

Es la ruta del archivo que se quiere visualizar.

### 3 Código completo del archivo

```
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

### 4 Función general del archivo

El archivo `base.py` define una clase llamada `Base`.

Esta clase actúa como clase base para los modelos ORM de SQLAlchemy.

En SQLAlchemy, un modelo ORM es una clase Python que representa una tabla de base de datos.

Por ejemplo:

|              |                    |
|--------------|--------------------|
| Clase Python | → Tabla PostgreSQL |
| Event        | → events           |
| Rule         | → rules            |
| Alert        | → alerts           |

Para que SQLAlchemy pueda reconocer estas clases como modelos de base de datos, deben heredar de una base declarativa.

En este proyecto, esa base declarativa es:

```
Base
```

La función de este archivo es centralizar esa base común para que todos los modelos compartan la misma estructura de registro interno.

### 5 Estructura general del archivo

El archivo tiene dos bloques:

```
from sqlalchemy.orm import DeclarativeBase
```

Importa la clase `DeclarativeBase` desde SQLAlchemy.

```
class Base(DeclarativeBase):
    pass
```

Define la clase `Base`, que hereda de `DeclarativeBase`.

Visualmente:

```
base.py
├─ Importación de DeclarativeBase
└─ Definición de clase Base
```

### 6 Análisis línea por línea

#### Importación de `DeclarativeBase`

```
from sqlalchemy.orm import DeclarativeBase
```

Esta línea importa `DeclarativeBase` desde el módulo ORM de SQLAlchemy.

Desglose:

```
from sqlalchemy.orm
```

Indica que la importación se realiza desde la parte ORM de SQLAlchemy.

ORM significa:

```
Object Relational Mapper
```

Es decir, una herramienta que permite relacionar clases Python con tablas de una base de datos relacional.

```
import DeclarativeBase
```

Importa la clase `DeclarativeBase`.

`DeclarativeBase` se utiliza en SQLAlchemy 2.x para crear una clase base declarativa moderna.

Esta clase base sirve como punto común para definir modelos ORM.

---

## Definición de la clase `Base`

```
class Base(DeclarativeBase):
```

Esta línea define una clase llamada `Base`.

Desglose:

```
class
```

Palabra clave de Python para definir una clase.

```
Base
```

Nombre de la clase.

Se usa el nombre `Base` por convención. En muchos proyectos con SQLAlchemy, la clase base de los modelos se llama así.

```
(DeclarativeBase)
```

Indica que `Base` hereda de `DeclarativeBase`.

Esto significa que `Base` recibe el comportamiento necesario para actuar como clase base declarativa de SQLAlchemy.

```
:
```

Marca el inicio del bloque de código de la clase.

---

## Qué significa heredar de `DeclarativeBase`

Cuando se define:

```
class Base(DeclarativeBase):
```

se está creando una clase personalizada que SQLAlchemy usará como base para los modelos.

Después, los modelos pueden definirse así:

```
class Event(Base):  
    ...
```

```
class Rule(Base):  
    ...
```

```
class Alert(Base):  
    ...
```

Al heredar de `Base`, esos modelos quedan registrados dentro del sistema declarativo de SQLAlchemy.

Esto permite que SQLAlchemy conozca:

- Qué clases representan tablas.
- Qué columnas tiene cada tabla.
- Qué nombres de tabla se han definido.
- Qué metadatos forman parte del esquema.

---

## Cuerpo de la clase

```
pass
```

La palabra `pass` indica que la clase no añade ningún comportamiento adicional.

En Python, una clase no puede quedar vacía. Si no se quiere definir nada dentro de ella, se usa `pass`.

Desglose:

```
pass
```

Instrucción nula de Python.

No ejecuta ninguna acción, pero permite que la clase sea sintácticamente válida.

En este caso significa:

```
La clase Base hereda todo el comportamiento de DeclarativeBase y no añade nada más.
```

Esto es suficiente para que los modelos del proyecto puedan heredar de `Base`.

---

## Resultado final del archivo

Después de cargar este archivo, queda disponible la clase:

```
Base
```

Esta clase puede ser importada por los modelos del proyecto.

Ejemplo conceptual:

```
from app.db.base import Base  
  
class Event(Base):  
    ...
```

La relación es:

```
DeclarativeBase  
    ↓  
Base
```

## 7 Relación con el flujo técnico del laboratorio

`base.py` no conecta directamente con PostgreSQL ni ejecuta consultas.

Su función es estructural: permite definir modelos ORM reutilizables.

La relación técnica sería:

```
base.py
  ↓
Base
  ↓
modelos SQLAlchemy
  ↓
Event / Rule / Alert
  ↓
SQLAlchemy metadata
  ↓
Alembic / PostgreSQL
```

Dentro del flujo general del SIEM, participa de forma indirecta:

```
Evento recibido
  ↓
Endpoint crea objeto Event
  ↓
Event hereda de Base
  ↓
SQLAlchemy sabe mapear Event a una tabla
  ↓
El evento se guarda en PostgreSQL
```

Sin `Base`, los modelos no tendrían una clase declarativa común y SQLAlchemy no podría gestionarlos correctamente como tablas ORM.

## 8 Errores típicos o puntos importantes

### La clase `Base` debe importarse en los modelos

Los modelos deben heredar de `Base`.

Ejemplo conceptual:

```
class Event(Base):
    ...
```

Si un modelo no hereda de `Base`, SQLAlchemy no lo tratará como modelo ORM declarativo.

### `Base` no representa una tabla

La clase `Base` no es una tabla de la base de datos.

Es una clase base para definir otras clases que sí representarán tablas.

Es decir:

```
Base → estructura común
Event → tabla real
```

```
Rule → tabla real
Alert → tabla real
```

## **pass no significa que el archivo no sirva**

Aunque el archivo parezca demasiado simple, cumple una función importante.

La lógica está heredada de `DeclarativeBase`.

Por eso no hace falta añadir métodos o atributos dentro de `Base`.

## **Relación con Alembic**

Alembic necesita conocer los metadatos de los modelos para generar o comparar migraciones.

Estos metadatos se construyen a partir de las clases que heredan de `Base`.

Por tanto, `Base` también participa indirectamente en la gestión de migraciones.

## **SQLAlchemy 2.x**

El uso de:

```
DeclarativeBase
```

corresponde al estilo moderno de SQLAlchemy 2.x.

En versiones anteriores era más habitual encontrar:

```
declarative_base()
```

Ejemplo antiguo:

```
Base = declarative_base()
```

En este proyecto se usa la forma moderna:

```
class Base(DeclarativeBase):
    pass
```

## **9 Comandos útiles relacionados**

Probar que `Base` puede importarse:

```
docker exec -it siem-api python -c "from app.db.base import Base; print(Base)"
```

Comprobar modelos que heredan de `Base`:

```
docker exec -it siem-api python -c "from app.models.event import Event; from app.models.rule import Rule; from app.models.alert import Alert; print(Event, Rule, Alert)"
```

Comprobar las tablas registradas en los metadatos de SQLAlchemy:

```
docker exec -it siem-api python -c "from app.db.base import Base; import app.models.event, app.models.rule, app.models.alert; print(Base.metadata.tables.keys())"
```

Comprobar tablas existentes en PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "\dt"
```

Ver migraciones de Alembic:

```
ls backend/alembic/versions
```

Comprobar versión actual de migración:

```
docker exec -it siem-api alembic current
```